

Contents

1. Introduction of Software Engineering
2. Need of Software Engineering
3. Characteristics of Good Software
4. Nature of Software
5. Software Process
6. SDLC Models



What is Software Engineering?

- **Software:** It is a collection of integrated programs. Means it carefully-organized instructions and code written by developers on any of various particular computer languages.
- **Engineering:** It is the application of scientific and practical knowledge to invent, design, build, maintain and improve frameworks, processes, etc.
- **Software Engineering:** It is an engineering branch related to the evolution of software product using well-defined scientific principles, techniques and procedures.
The result of software engineering is an effective and reliable software product.

Need of Software Engineering

1. **Handling Big Projects:** Corporation must use SE to handle large projects without any issues.
2. **To manage the cost:** Software engineering programmers plan everything and reduce all those things that are not required.
3. **To decrease time:** It will save a lot of time if you are developing software using a software engineering technique.
4. **Reliable software:** It is the company's responsibility to deliver software products on schedule and to address any defects that may exist.
5. **Effectiveness:** Effectiveness results from things being created in accordance with the software standards.

Characteristics of Good Software

Operational	Transitional	Maintenance
Budget	Interoperability	Flexibility
Efficiency	Reusability	Maintainability
Usability	Portability	Modularity
Dependability	Adaptability	Scalability
Correctness		
Functionality		
Safety		
Security		

Nature of Software

1. System Software:

- System software is software designed to provide a platform for other software's.
- It is a interaction  between hardware & application software.
- **Examples :** Operating Systems like macOS, Linux, Android and Microsoft Windows.

2. Application Software:

- Application Software is a computer program designed to carry out a specific task as per user & business need.
- **Examples:** Social medias apps, Gaming apps, Word processing apps, Multimedia apps, Banking apps, Shopping apps, Booking apps etc.



Nature of Software

3. Engineering and Scientific Software:

- This software is used to facilitate the engineering function and task take real time.
- It has very high accuracy, complex formula evolution & data analysis.
- **Examples:** Weather prediction apps, Stock Market apps, Stress Analysis, Body measurement apps

4. Embedded Software:

- Embedded software resides within the system or product and is used to implement and control feature and function for the end-user and for the system itself.
- **Example:** Switches, Routers, Digital camera, Washing machine functionalities, Traffic control etc.

Nature of Software

5. Web Applications:

- It is a client-server computer program which the client runs on the web browser.
- Web apps can be little more than a set of linked hypertext files that present information using text and limited graphics.
- **Examples:** Online forms, Shopping carts, Gmail, Yahoo, Photo editing, File conversion etc.

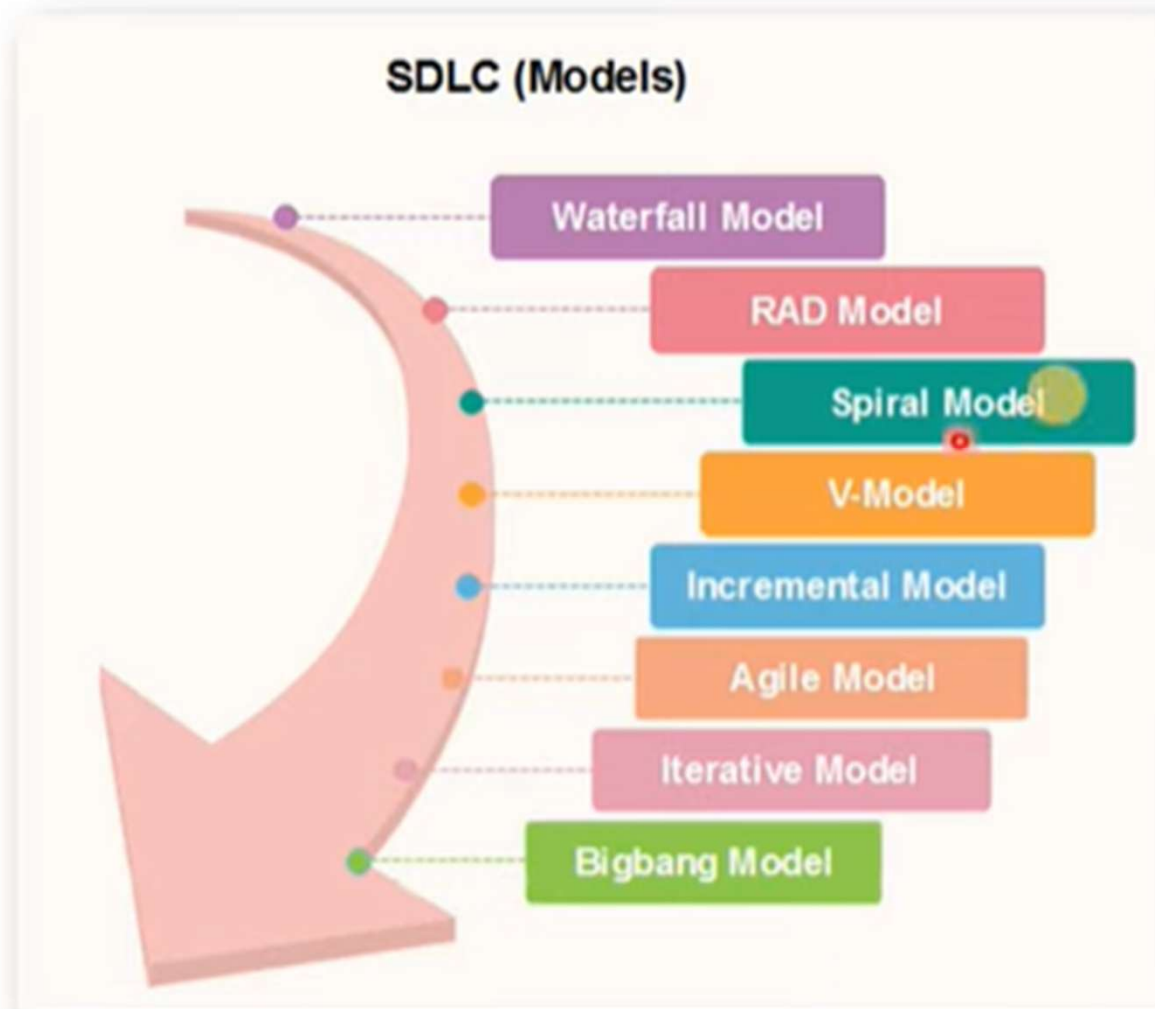
6. Artificial Intelligence Software:

- It makes use of a nonnumerical algorithm to solve a complex problem.
- Application within this area includes robotics, expert system, pattern recognition, artificial neural network, theorem proving and game playing.
- **Examples:** Google Cloud, Azure studio, Tensor Flow, Salesforce etc.

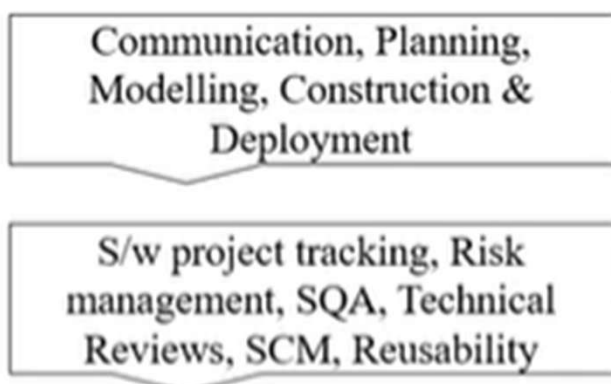
Software Process

- A software process is the set of activities and associated outcome that produce a software product.
 - Software engineers mostly carry out these activities.
-
1. **Software Specifications:** The functionality of the software and constraints on its operation must be defined.
 2. **Software Development:** The software to meet the requirement must be produced.
 3. **Software Validation:** The software must be validated to ensure that it does what the customer wants.
 4. **Software Evolution:** The software must evolve to meet changing client needs.
-

Software Engineering – SDLC Models



Generic Process Model



- **Software process** means to develop quality products by using technical & management rules.
- **The generic process model** is an description of the software development process.
- It is used in most software models since it provides a base for them.

Software process

Process framework

Umbrella activities

framework activity # 1

software engineering action #1.1

Task sets

work tasks
work products
quality assurance points
project milestones

⋮

software engineering action #1.k

Task sets

work tasks
work products
quality assurance points
project milestones

⋮

framework activity # n

software engineering action #n.1

Task sets

work tasks
work products
quality assurance points
project milestones

⋮

software engineering action #n.m

Task sets

work tasks
work products
quality assurance points
project milestones

Process Framework Activities



1. Communication:



- The software development starts with the communication between customer and developer.
- Developer gather requirements of the project with the users.
- Communication with consumers and stakeholders to determine the system's objectives, software's requirements & features.

2. Planning:

- It consists of complete estimation, scheduling for project development and tracking.
- Discuss work plan, technical risk, list of resource requirements, work produced & work schedule.

Process Framework Activities

3. Modelling:

- Develop a practical model to get a better understanding of the project.
- It consists of complete requirement analysis and design of the project like algorithm, flowchart.
- The algorithm is the step-by-step solution of the problem.
- The flow chart shows a complete flow diagram of a program.
- If changes are required, we implement them in this step.

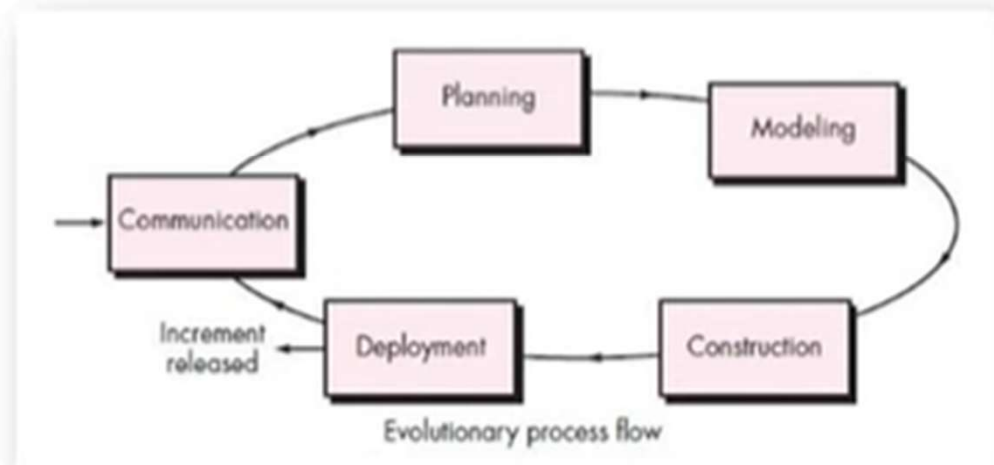
4. Construction:

- Construction consists of code generation and the testing part.
 - Coding part implements the design details using an appropriate programming language.
 - Testing is to check whether the flow of coding is correct or not.
 - Testing also check that the program provides desired output.
-

Process Framework Activities

5. Deployment:

- Deployment step consists of delivering the product to the customer and take feedback from them.
- If the customer wants some corrections or demands for the additional capabilities, then the change is required for improvement in the quality of the software.



Umbrella Activities

➤ Activities that occur throughout a software process for better management and tracking of the project.

1. Software Project Tracking and Control:

- Compare the progress of the project with the plan and take steps to maintain a planned schedule.

2. Risk Management:

- Analyze any potential risks that may have an impact on the software product's quality and outcome.

3. Software Quality Assurance (SQA):

- These are activities required to maintain software quality.
- Perform actions to ensure the product's quality.

Umbrella Activities

4. Technical Reviews :

- Assessment of errors and correction done at each stage of activity.

5. Software Configuration Management (SCM):

- Managing of configuration process when any change in the software occurs.

6. Reusability Management:

- Reusable work items should be backed up, and reusable software components should be achieved.

7. Work Product Preparation and Production:

- The activities to create models, documents, logs, forms and lists are carried out.

About SDLC Process

- SDLC represents the process of developing quality software as per customer requirements.

➤Need of SDLC Process:

1. Improved relation with client & development team.
2. It offers basic project planning, scheduling & cost estimation.
3. Provide standard set of activities & SE rules.
4. Increase & enhance development speed.
5. Maintain project tracking & control.
6. Decrease any type of project risk.
7. Decide entry & exit criteria of each phase.



SDLC Phase 1: Requirement Analysis & Planning

- Phase conduct by Product manager, Business Analyst or senior members in team.
- To understand the Aim, Purpose & Exact requirements of the customer.
- Conduct market research, customer interviews & surveys.
- Software Requirement Specification (SRS) document is created.
- Identify risk assessment in project.
- Planning of Project schedule, Cost estimation & other recourses.

Example: Online Shopping Application

Customer requirements like:

- Categories like Ladies wear, Gents wear module, Grocery module, Shopping cart module, My order module, Discount module, Notification module.
- Good & Attractive user interface.

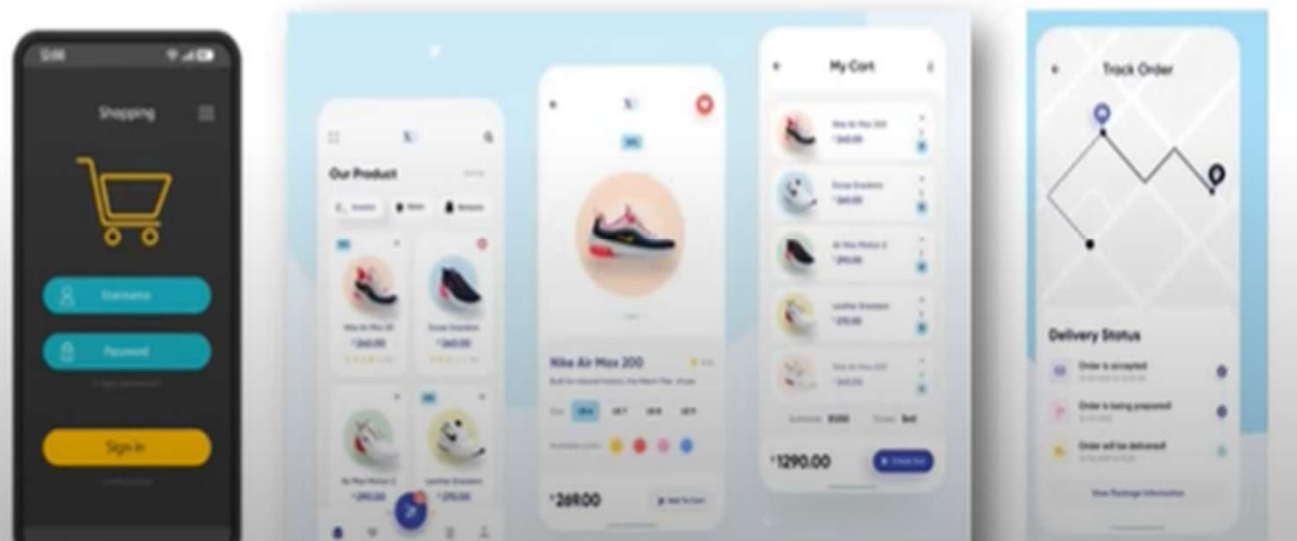


SDLC Phase 2: Design

- Phase conduct by Product manager & UI / UX Designer team.
- High level design include Software Architecture like brief description of each module, Flow Diagram, Database tables, Interface relationships & Algorithm.
- Low level design include Input & Output of each module, Rough paper design, Detail of Interface, Listing error messages etc.
- Documented in DDS “Design Document Specification”.

Example:

Online Shopping Application



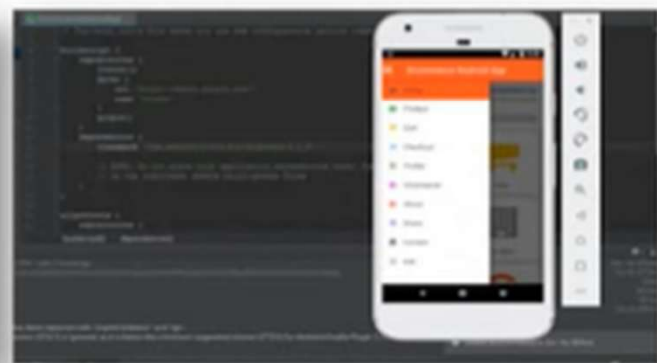
SDLC Phase 3: **Implementation**

- Phase conduct by Developer or Programmer.
- Must follow predefine coding guidelines or discuss with management team.
- Used suitable Programming Languages like C, C++, JAVA, ANDROID, PYTHON, PHP etc.
- Used suitable Databases like ^oSQL, ORACLE etc.
- Programming tools like Compilers, Interpreters, Debuggers etc.

Example:

Online Shopping Application

```
void customerDetails()
{
    cout << "Enter your name: ";
    string customer_name;
    getline(cin, customer_name);
    cout << "WELCOME ";
    for(int i = 0; i < customer_name.length(); i++)
    {
        cout << char(toupper(customer_name[i]));
    }
    cout << "\n";
}
```



SDLC Phase 4: **Testing**

- Phase conduct by QA Team or Tester.
- It perform Integration Testing, Black Box Testing, White Box Testing, System Testing, Regression Testing Performance Testing & Acceptance Testing etc.
- The testing team starts testing the functionality of the entire system.
- If find some bugs/defects which report to developers then Development team fixes the bug and send back to QA.
- This is done to verify that the entire application works according to the customer requirement.

Example:

Online Shopping Application



SDLC Phase 5: **Deployment & Maintenance**

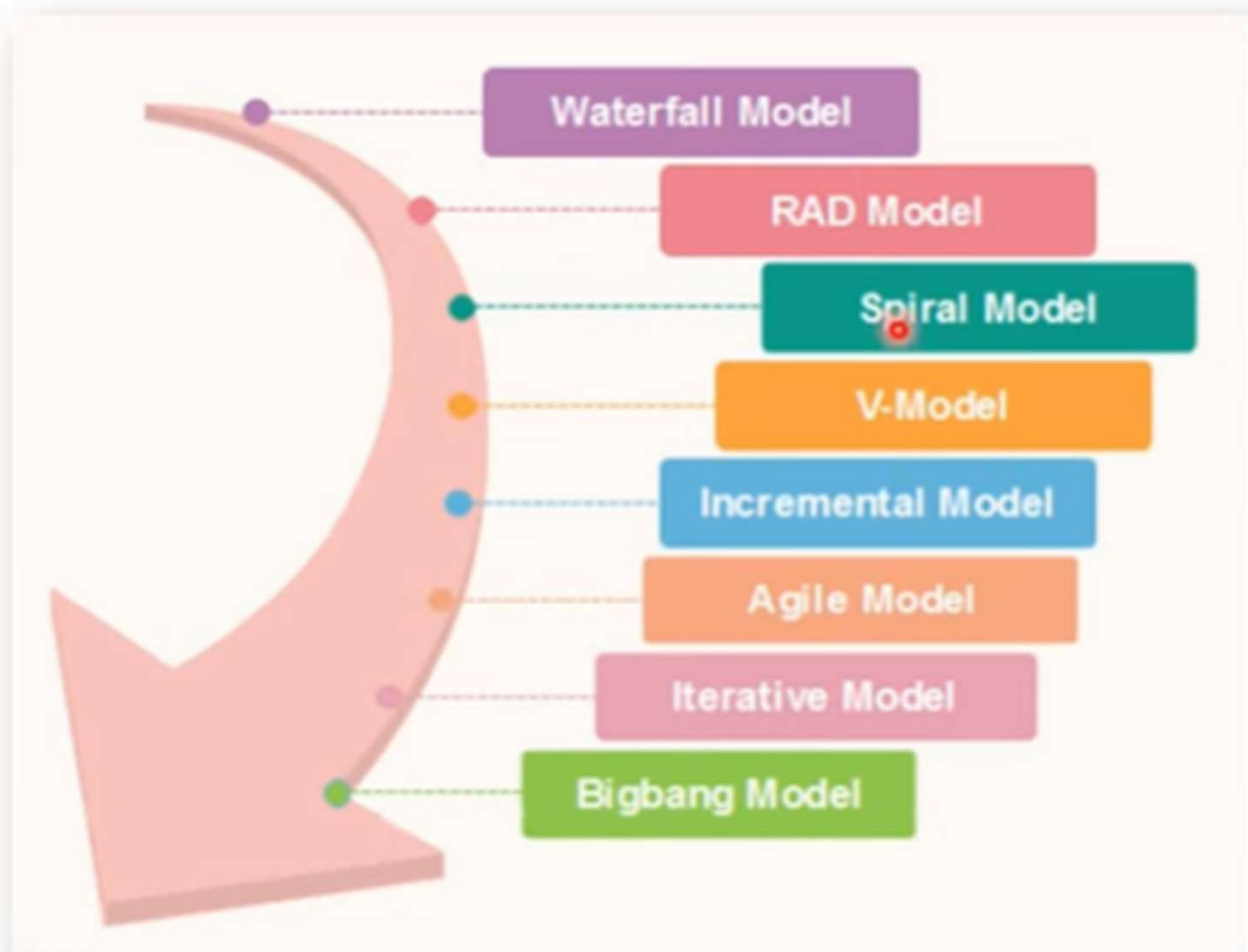
- Phase conduct by Project Manager, Operation or Production support team.
- Once the software is certified and no bugs or errors are stated, then final software is release in market or deployed in customer environment.
- If any issue comes up and needs to be fixed or any updating is to be done is taken care by the Support Managers in Maintenance phase.

Example:

Online Shopping Application



SDLC Models

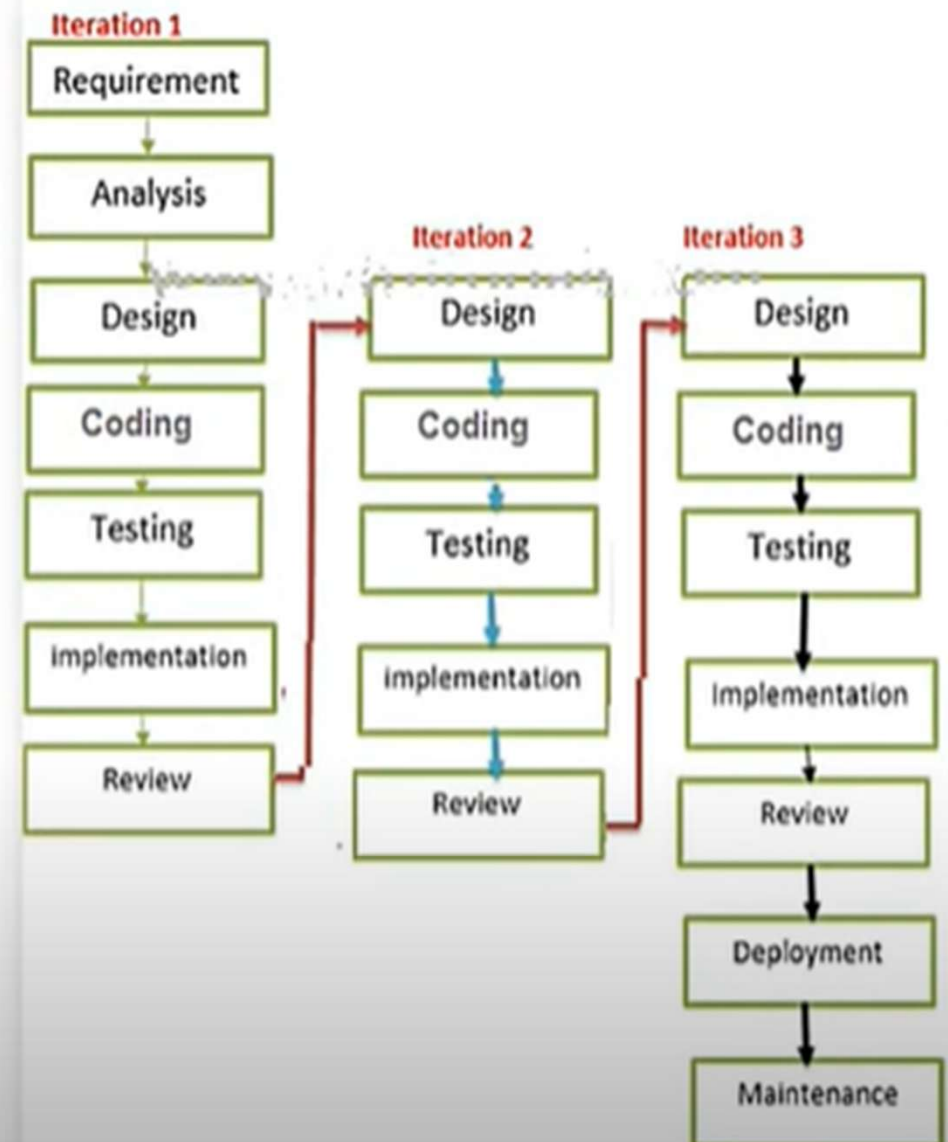


Iterative Model

➤SDLC Cycle start.

- Start with some Requirements & Analysis.
- Develop first version of S/W.
- If any changes, new version created.
- Again If any changes, new version created.
- Called as Iterations.
- After finalized, Deploy the product.

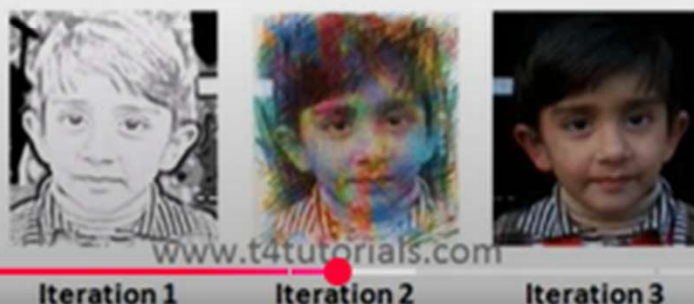
➤SDLC Cycle completed.



When to use Iterative Model?

1. When requirements are defined clearly and easy to understand.
2. When there is a requirement of changes in future.
3. When the software application is large.
4. A new technology is being used and is being learnt by the development team while working on the project.
5. There are some high-risk features and goals which may change in the future.

Example for just understanding:



Iterative Model Phases

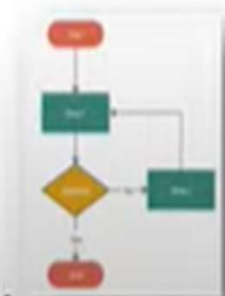
1. Requirement Gathering & Analysis:

- The aim of this phase is to understand the exact requirements of the customer.
- The system analyst analyses the requirements that whether the requirement can be fulfilled or not.
- If customer requirements can be fulfilled than whether these requirements are fulfilled within the budget or out of the budget
- If all okay, Then collected requirements are then planned accordingly for developing the system.



2. Design Phase:

- It design the overall software architecture together with high level and detailed design.
- Designer design Algorithm, Data Flow diagram, Activity diagram, Class diagram, State Transition diagram, Database design etc.



Iterative Model Phases



3. Coding Phase:

- In this phase software design is translated into specific source code.
- Programmers use suitable programming languages, frameworks & databases.

4. Testing Phase:

- In this phase, Tester perform all the testing activities to make sure that the system meets the client requirements or not.
- There are many test methods, but the most common are white box, black box and grey box test methods.



Iterative Model Phases



5. Review Phase:

- The review phase is where the software is estimated and checked as per the current requirement.
- Then, further requirements are reviewed discussed and reviewed to propose for an update in the next iteration.



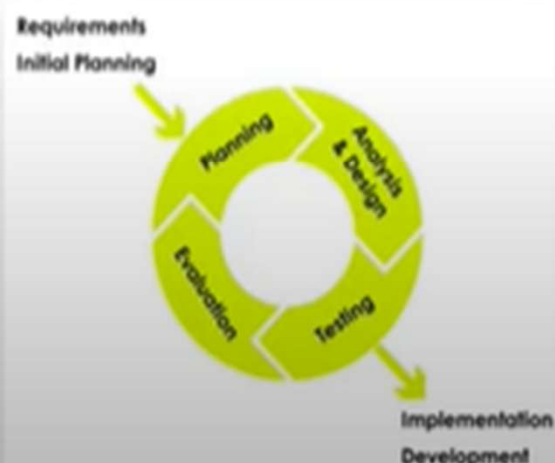
6. Deployment & Maintenance Phase:

- After completing, The product is deployed in the customer environment.
- After deployment of the software in the working environment there may be some bugs, some errors or new updates are required that solved in Maintenance Phase.



Advantages of Iterative Model

1. Testing and debugging during smaller iteration is easy.
2. A Parallel development can plan.
3. It is easily acceptable to ever-changing needs of the project.
4. Risks are identified and resolved during iteration.
5. Limited time spent on documentation and extra time on designing.



Disadvantages of Iterative Model

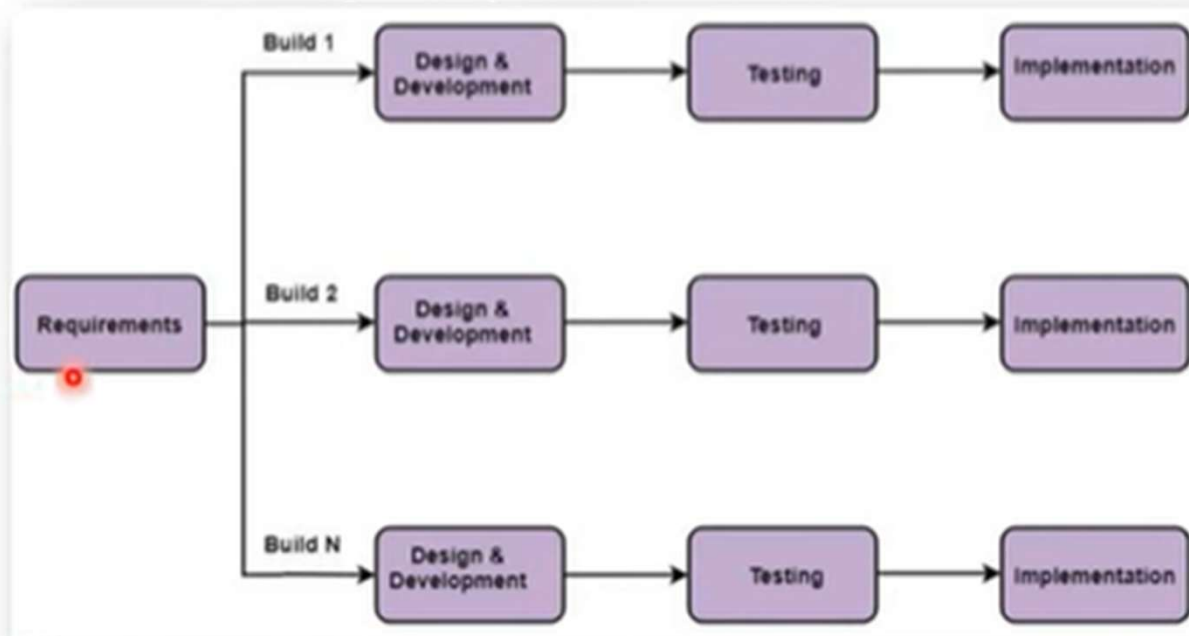


1. It is not suitable for smaller projects.
2. More Resources may be required.
3. Design can be changed again and again because of imperfect requirements.
4. Requirement changes can cause over budget.
5. Project completion date not confirmed because of changing requirements.



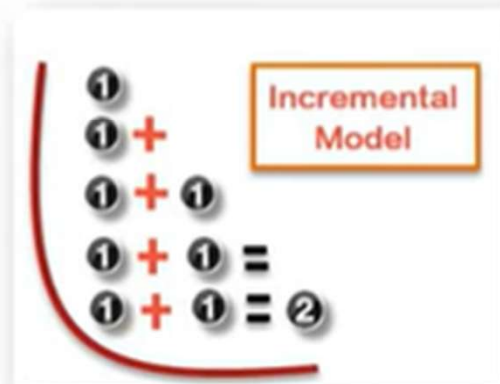
Incremental Process / Development Model

- In Incremental Model, requirements divided into multiple standalone modules of the software development cycle.
- Each module goes through the requirements, design, implementation and testing phases.
- Every subsequent release of the module adds function to the previous release.
- The process continues until the complete system achieved.



When to use Incremental Process Model?

1. Requirements are clearly specified, understood and are known up-front.
2. When the customer demands a quick release of the product.
3. You can develop prioritized requirements first.
4. A project has a long development schedule.
5. When Software team are not very well skilled or trained.
6. Lowers initial delivery cost.
7. Such methodology is more in use for web application and product based companies



Advantages of Incremental Process Model

1. The Client gets important functionality early.
2. Clients have a clear idea of the project.
3. Errors are easy to be recognized.
4. Easier to test and debug
5. Simple to manage risk because it handled during its iteration.
6. Changes are easy to implement.
7. It is flexible and less expensive to change requirements and scope.
8. Partial systems are successively built to produce a final total system.



Disadvantages of Incremental Process Model

1. Well defined module interfaces are needed.
2. A good team and proper planned execution are required because of its continuous iterations.
3. Problems might cause due to system architecture as such not all requirements collected up front for the entire software lifecycle.
4. Each iteration phase is rigid and does not overlap each other.

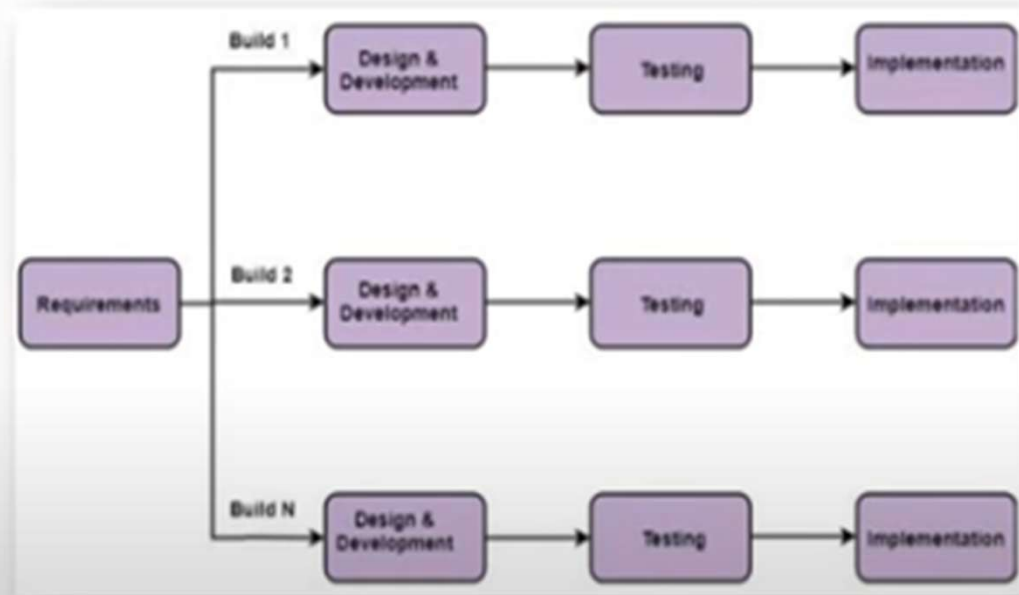
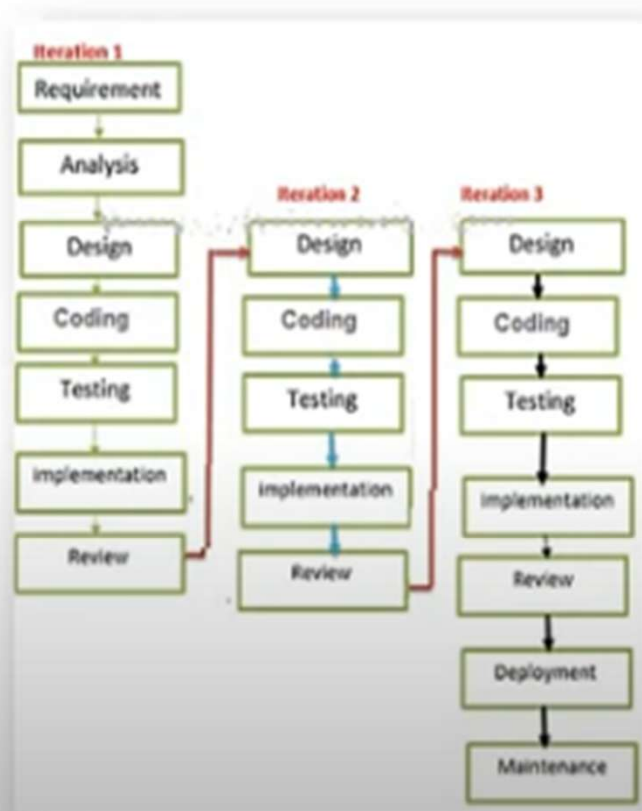


Evolutionary Process Model



- Evolutionary model is also referred to as the successive versions model.
- It is a combination of Iterative & Incremental model.

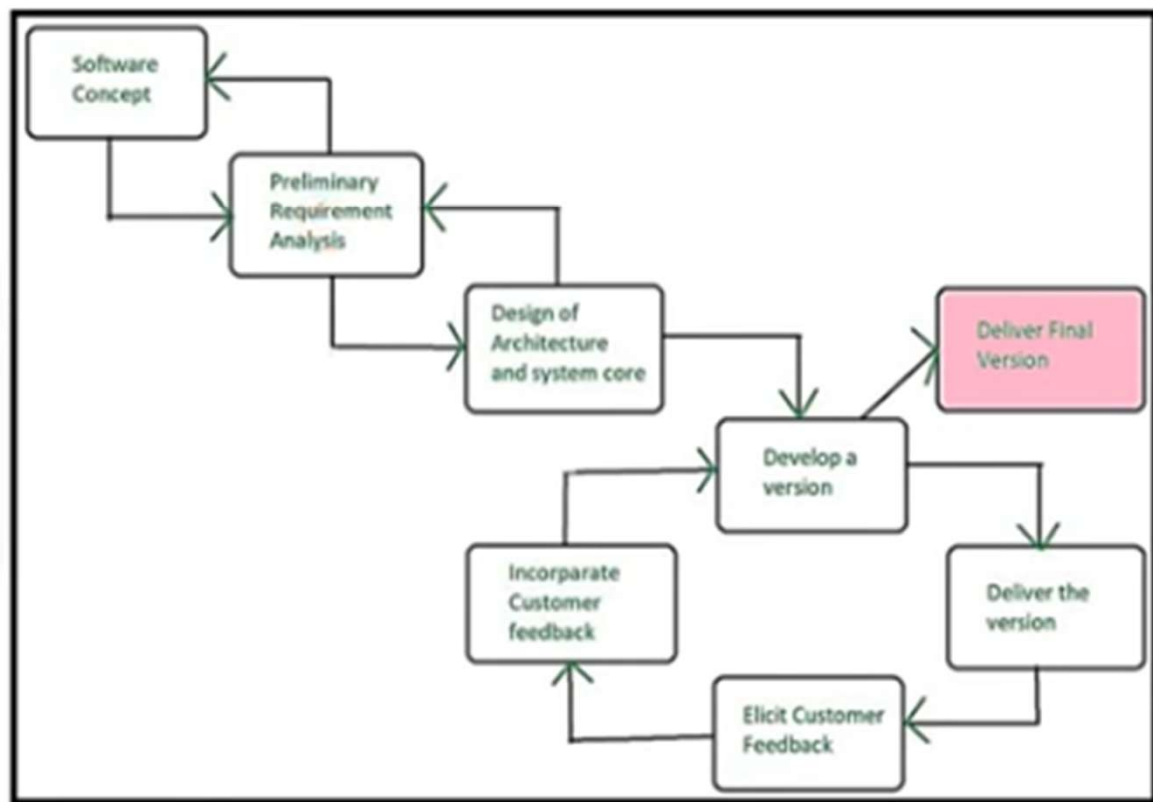
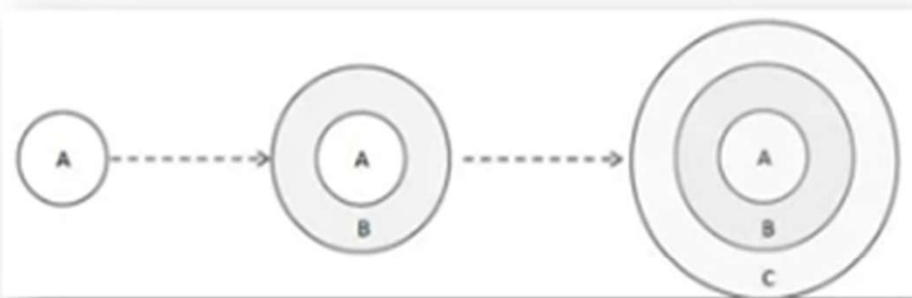
Revision:



Incremental Model

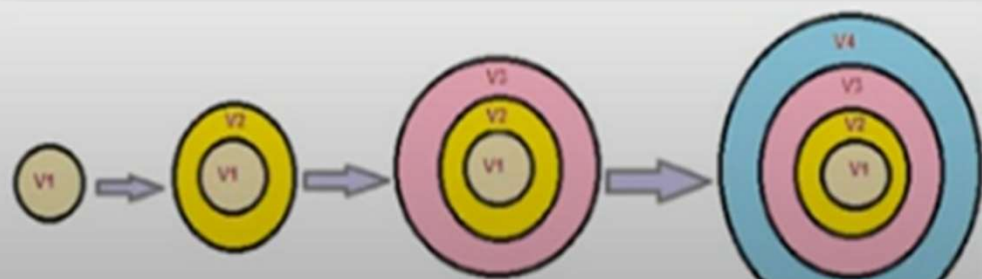
Evolutionary Process Model

- In Evolutionary model, the software requirement is first broken down into several modules
- That can be incrementally constructed means it take customer feedback of each module.
- Delivered product module by module to the customer.



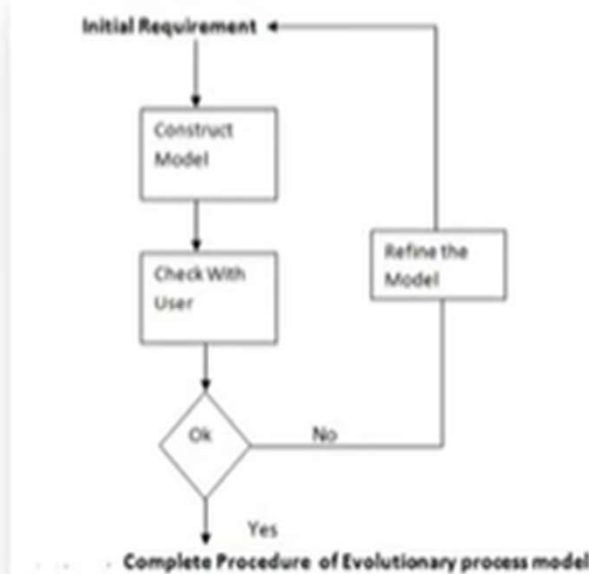
When to use Evolutionary Process Model?

1. Evolutionary model is commonly used when the customer wants to start using the core features instead of waiting for the full software.
2. It is used in large projects because of step by step development.
3. Customer requirements are not fixed. But clear concept.
4. Small changes required in separate modules.
5. Useful in object-oriented software development because all the development is divided into different units.



Advantages of Evolutionary Model

1. It is useful for very large products.
2. User has checked every stage during the development and it is helpful in achieving customer confidence.
3. User gets a chance to experiment with a partially developed software much before the complete version of the system is released.
4. It reduces the error because the core modules get tested thoroughly.
5. It also reduces the cost of development.



Disadvantages of Evolutionary Model

1. Constant and clear interaction between the development team members is required.
2. The delivery of full software can be late due to different changes by customers during development.
3. It is difficult to divide the problem into several parts.

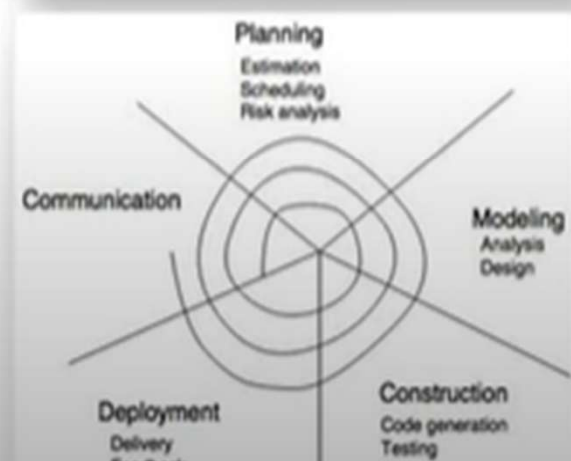


Types Evolutionary Process Model



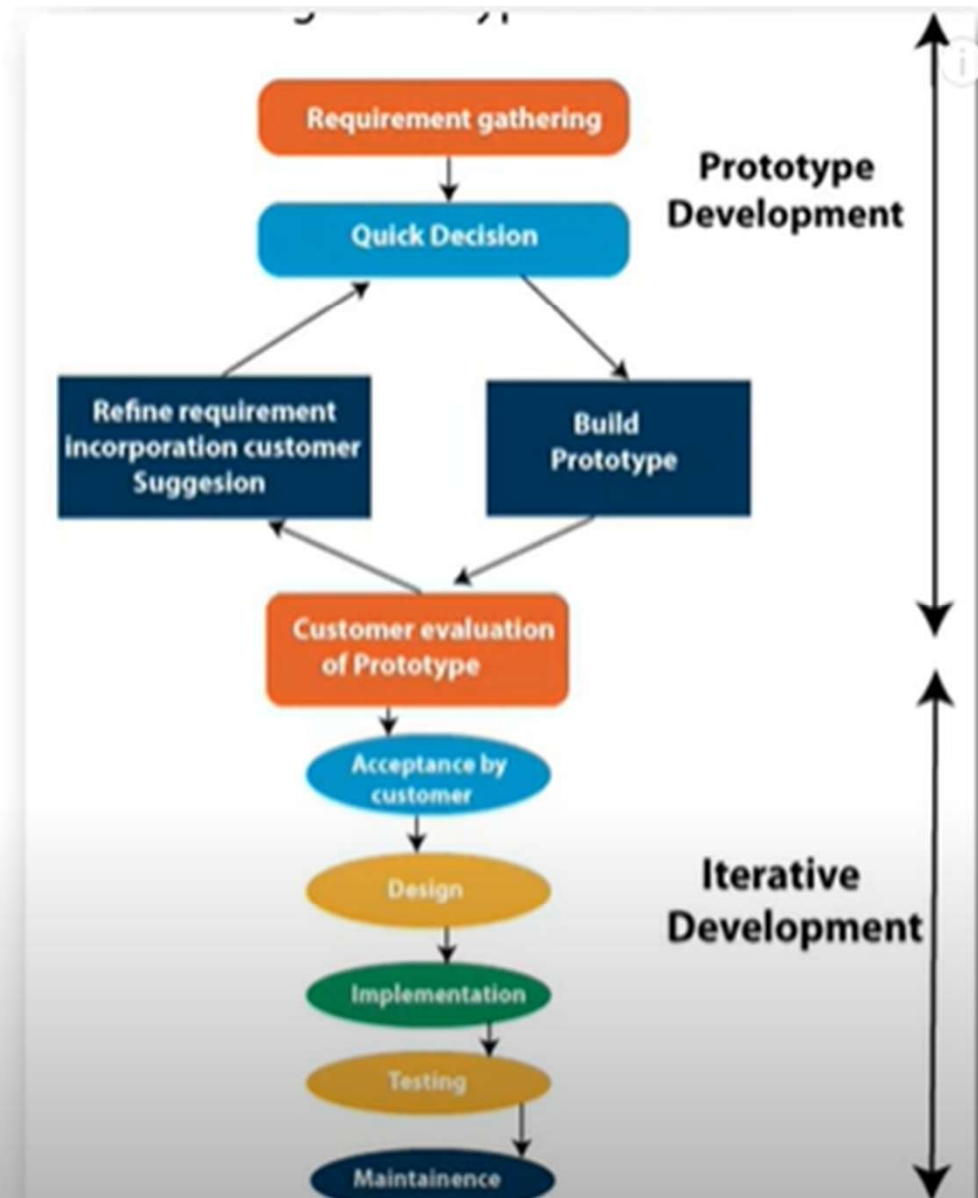
There are total three types of Evolutionary Model:

1. The prototyping model
2. The spiral model
3. Concurrent development model



Prototyping Model

- Prototype is not a complete product it is just functional replica or toy implementation of any idea, software or system.
- It is applied when customers do not know the exact project requirements.
- It generate before actual development of software.
- It is an iterative, trial and error method which takes place between developer and client.



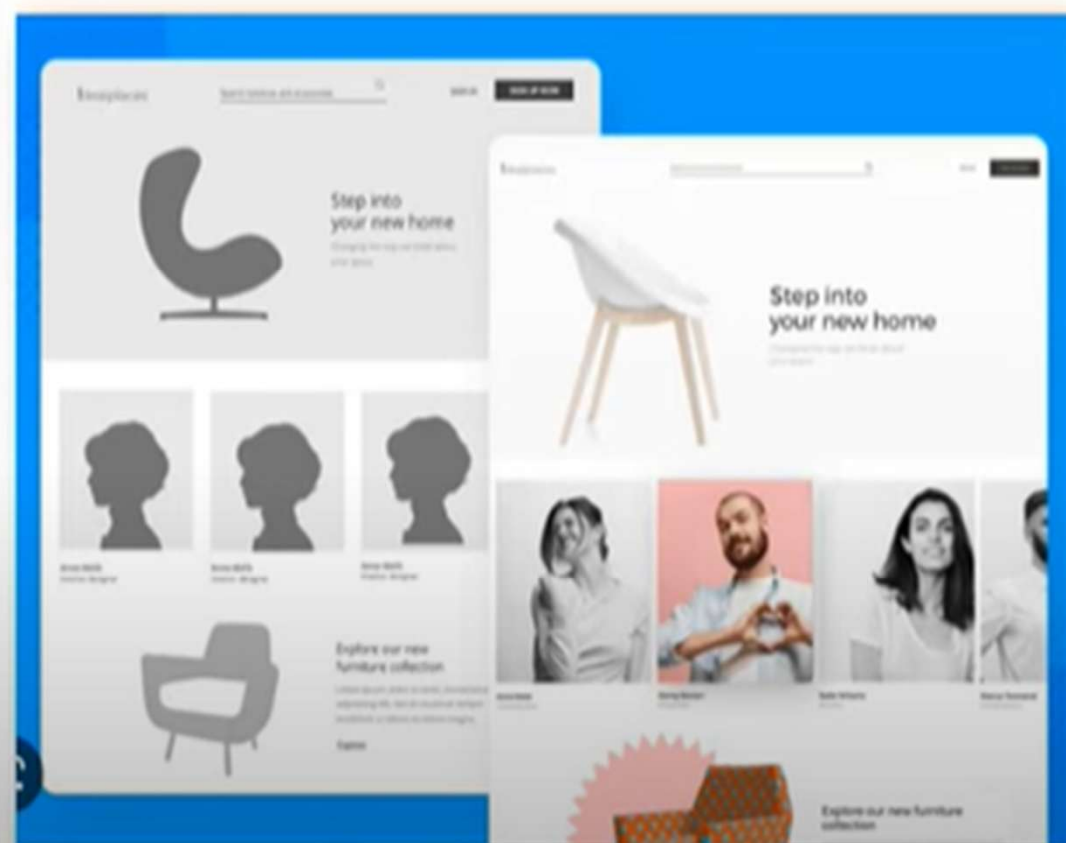
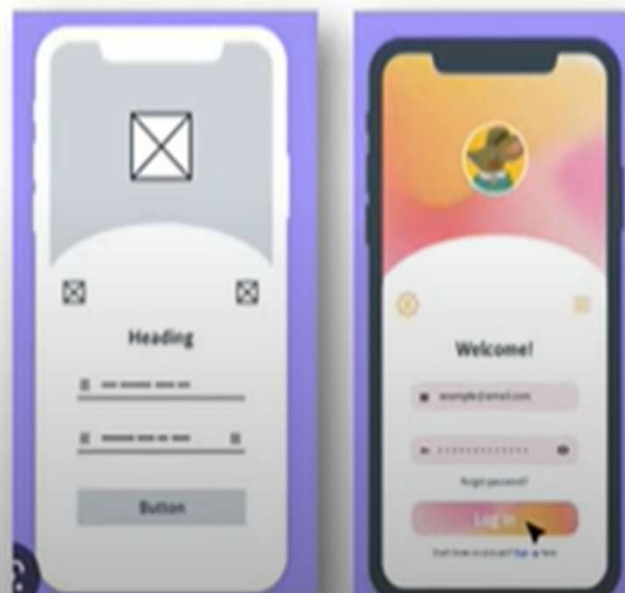
Example Prototype VS Actual

SE 6 : Incremental Process Model | Complete Explanation with Exa...



Prototype

Actual



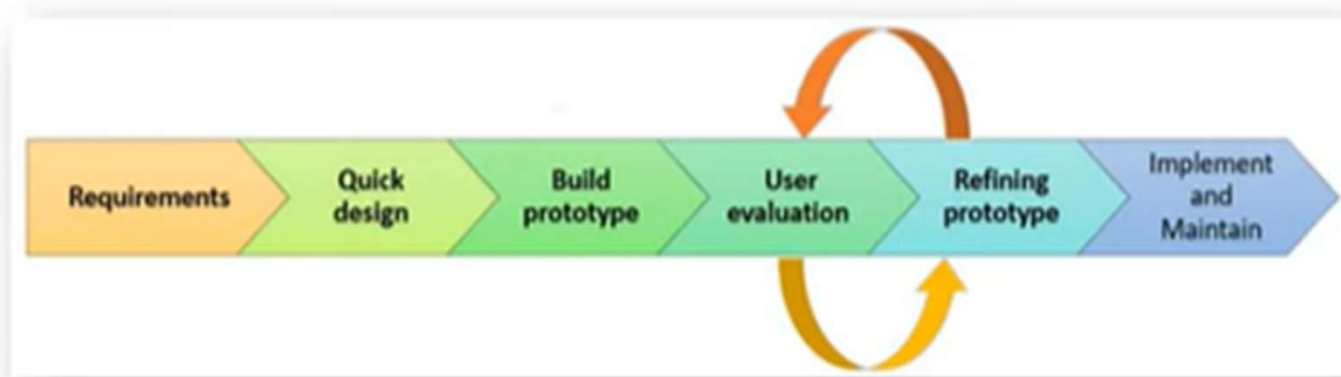
Advantages of Prototyping Model

1. Prototype model need not know the detailed input, output, processes, adaptability of operating system and full machine interaction.
2. Good where requirement are changing.
3. Customers are actively involved in development.
4. Prototypes can be changed and even discarded.
5. Errors can be detected much earlier as the system is made side by side.
6. Flexibility in design.
7. Helps team member to communicate effectively



Disadvantages of Prototyping Model

1. The client involvement is more and it is not always considered by the developer.
2. It is a slow process because it takes more time for development.
3. Many changes can disturb the rhythm of the development team.
4. Documentation is poor as the requirements change frequently.
5. It is a thrown away prototype when the users are confused with it.
6. Costly with respect to time & money.
7. Prototyping tools are expensive.



About Spiral Model

- Initially proposed by Boehm in 1986.
- Spiral Model is a **risk-driven software development process model.**
- It is a combination of Waterfall model, Iterative model & Prototyping model.
- Software is developed in a series of incremental releases as per each spiral.
- Example:** Microsoft, OS Versions, Gaming industry ex.
- Also called as Meta model.

➤ Spiral model divided into the four parts:

1. Planning
2. Risk Analysis
3. Engineering & Execution
4. Customer Evaluation



Spiral Model Phases

1. Planning: (Requirement Gathering & Analysis)

- Communication between customers & project head.
- Collect all the requirements from customers.
- Analyses estimated cost, schedule & required recourses.

2. Risk Analysis:

- Identification of all the potential risks.
- Risk mitigation strategy is planned for solving risks.
- Design a prototype of model.



Spiral Model Phases

3. Engineering & Execution:

- Actual development start.
- Designer design the product as per final prototype.
- Developer perform actual coding or implementation.
- Tester perform all testing methods.
- Deploy or Release product to the customer environment.

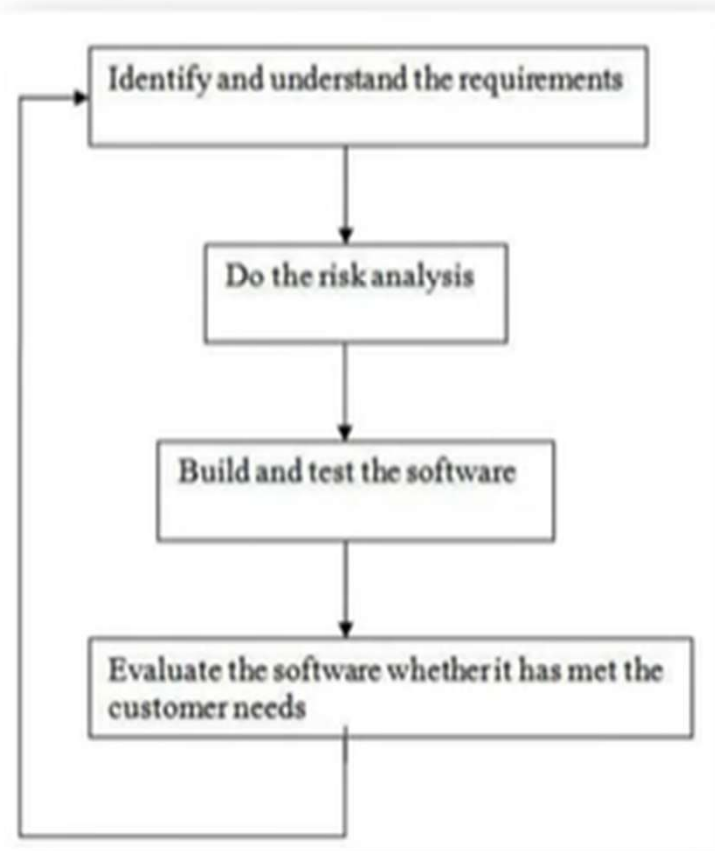
4. Evaluation:

- Take a feedback from customers.
- If customer want any changes, goes to next planning OR next spiral iteration.



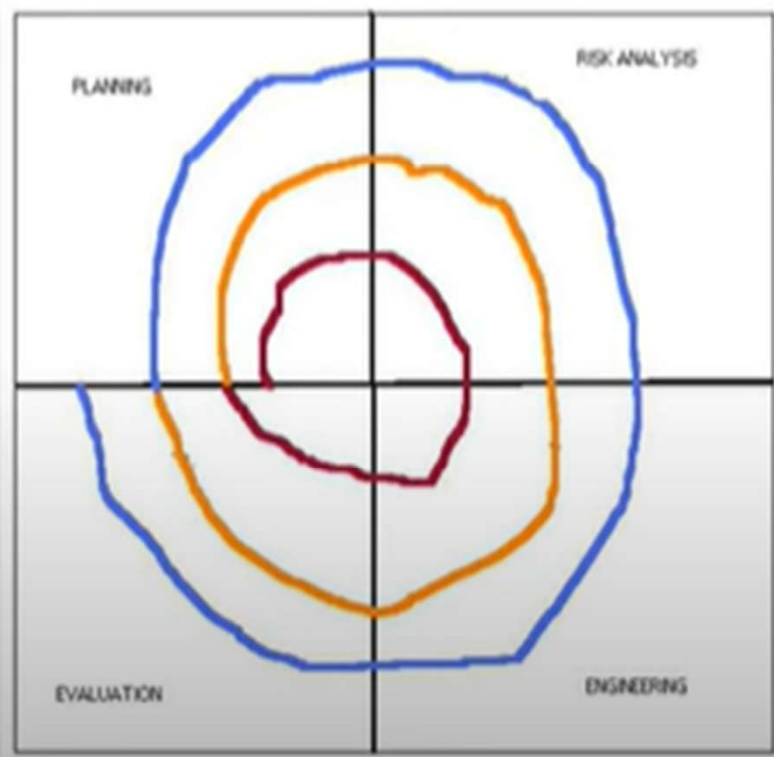
When to use Spiral Model?

1. When the project is large & high budget.
2. When requirements are unclear and complex.
3. Where the software needs continuous risk evaluation.
4. When creation of a prototype is applicable.
5. When changes may require at any time.
6. When long term project commitment is not feasible due to changes in economic priorities



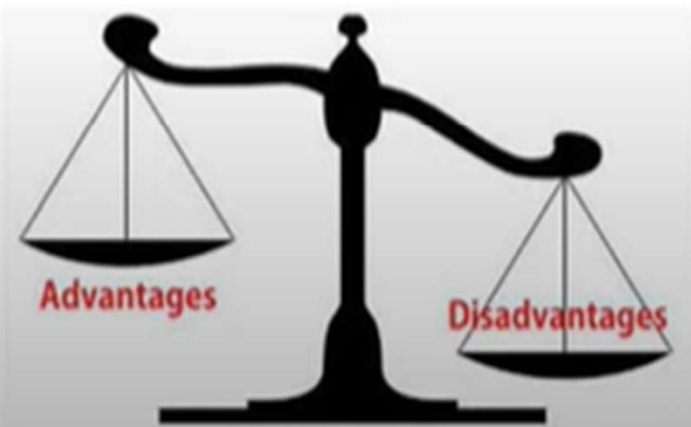
Advantages of Spiral Model

1. High amount of risk analysis.
2. Risky parts can be developed earlier which helps in better risk management.
3. Useful for large and mission-critical projects.
4. Allows extensive use of prototypes.
5. There is always a space for customer feedback
6. Changing requirements can be accommodated.
7. Development is fast.



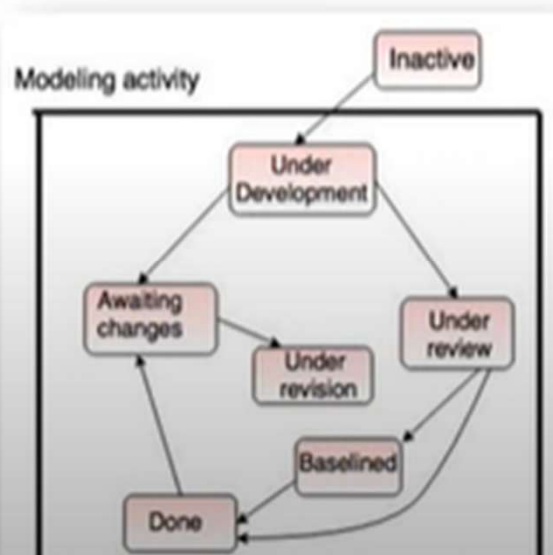
Disadvantages of Spiral Model

1. Risk analysis needed highly particular expertise.
2. Can be a costly model to use.
3. Doesn't work well for smaller projects.
4. Spiral Process is complex sometimes because Spiral may go infinitely.
5. Large number of spiral stages requires excessive documentation.



About Concurrent Model

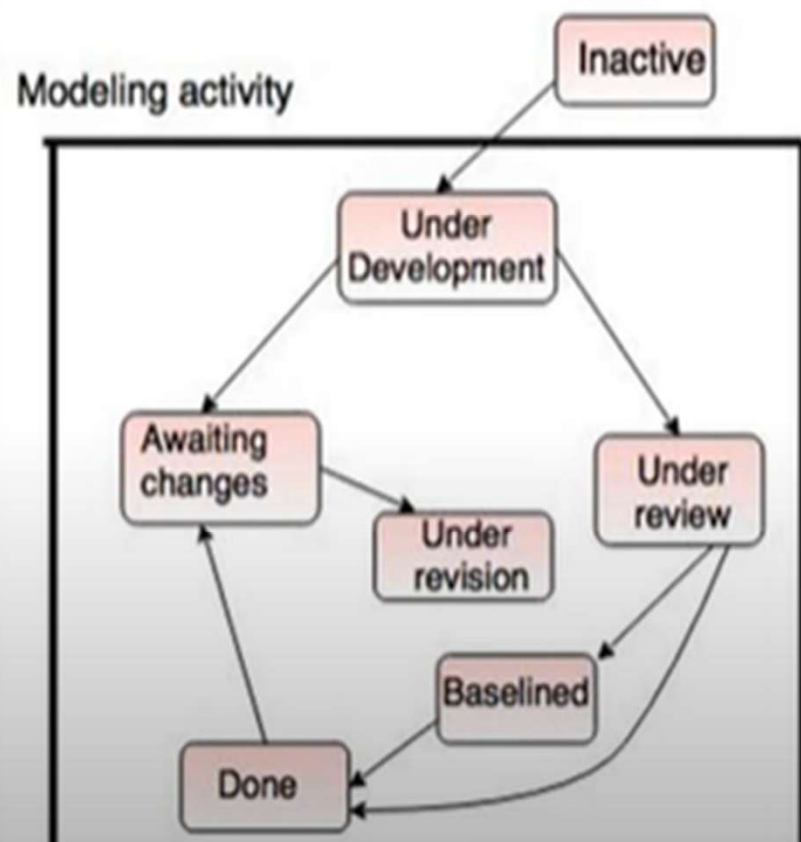
- It is a type of Evolutionary Model.
- Concurrent development model is also known as concurrent engineering.
- The term concurrent mean “done at the same time”.
- It used in all software development process model.
- Transition from state to state for each of the software engineering activities.



Concurrent Model Working

✓ SDLC Activities: Communication, Design, Coding, Testing, Deployment

- **Inactive**: No any activity / state performed.
- **Under Development**: Any activity performed.
- **Awaiting Changes**: If customer want any changes
- **Under review**: Testing activity start.
- **Under revision**: Do all required changes.
- **Baselined**: As per the SRS document
- **Done**: Project completed & Deployed.



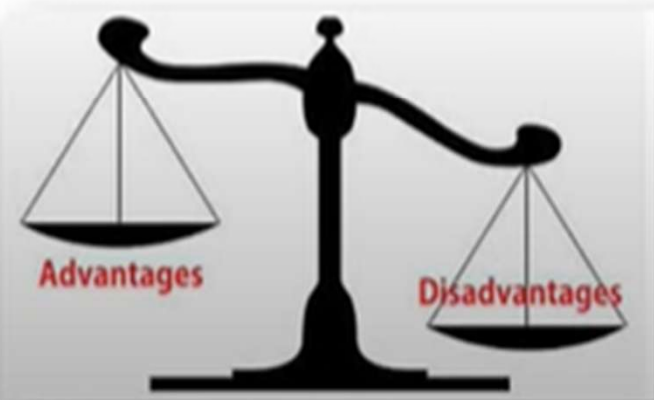
Advantages of Concurrent Model

1. This model is applicable to all types of software development processes.
2. It is easy for understanding and use.
3. It gives immediate feedback from testing.
4. It provides an accurate picture of the current state of a project.



Disadvantages of Concurrent Model

1. It needs better communication between the team members. This may not be achieved all the time.
2. It requires to remember the status of the different activities.



About Agile Model

➤ About Agile:

- Mostly used model in today's digital era.
- Agile means “The ability to respond to changes from requirements, technology & people”
- It is an incremental and iterative process of software development.

➤ Working with Example:

- Divides requirements into multiple iterations & provide specific functionality for the release.
- Delivers multiple software requirements.
- Each iteration lasts from two to three weeks.
- Direct collaboration with customers.
- Rapid project development.



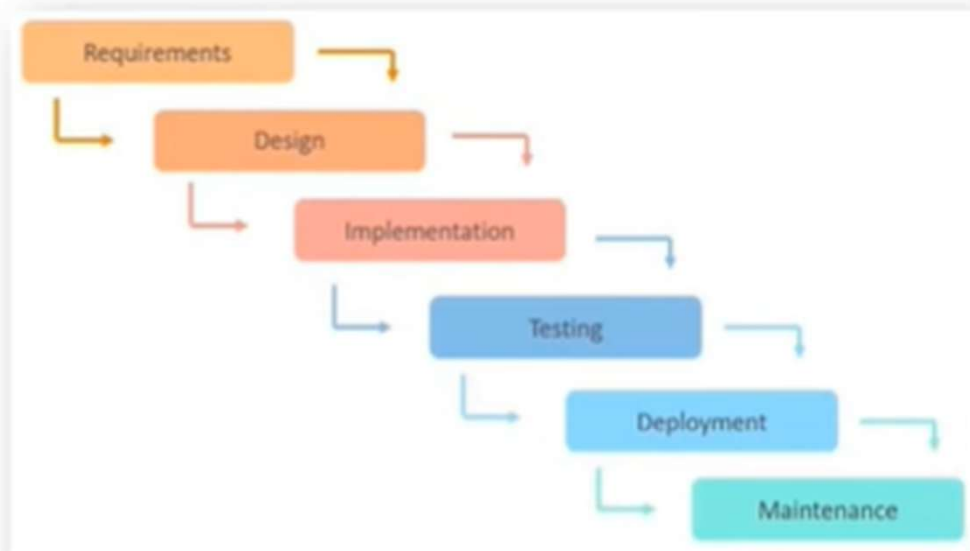
Traditional VS Agile Model Working with Example

For Example:

Instagram Social Application:

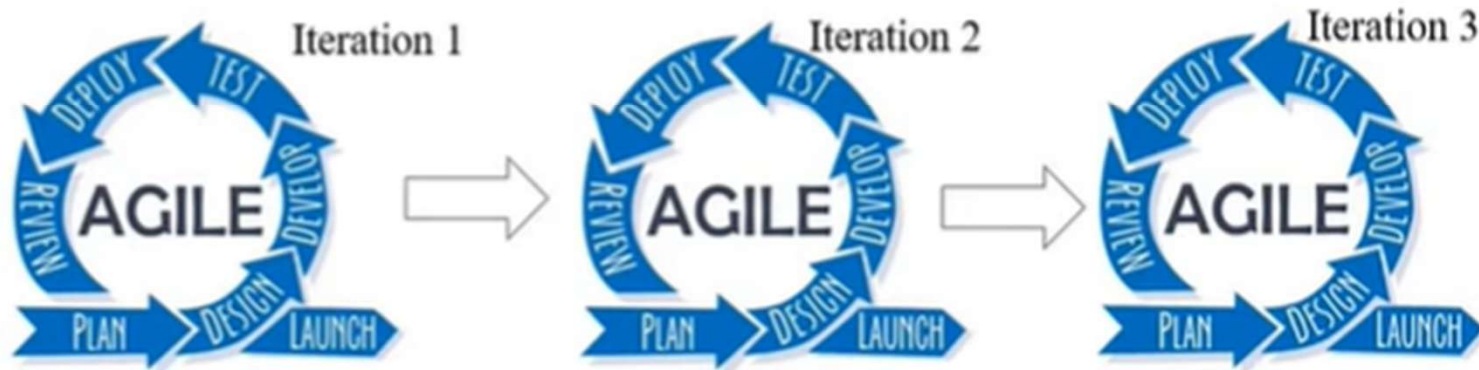
Requirements are:

1. Follow – Unfollow Option
2. Edit profile
3. Search
4. Messaging
5. Post photos
6. Upload story
7. To make reels
8. Go Live



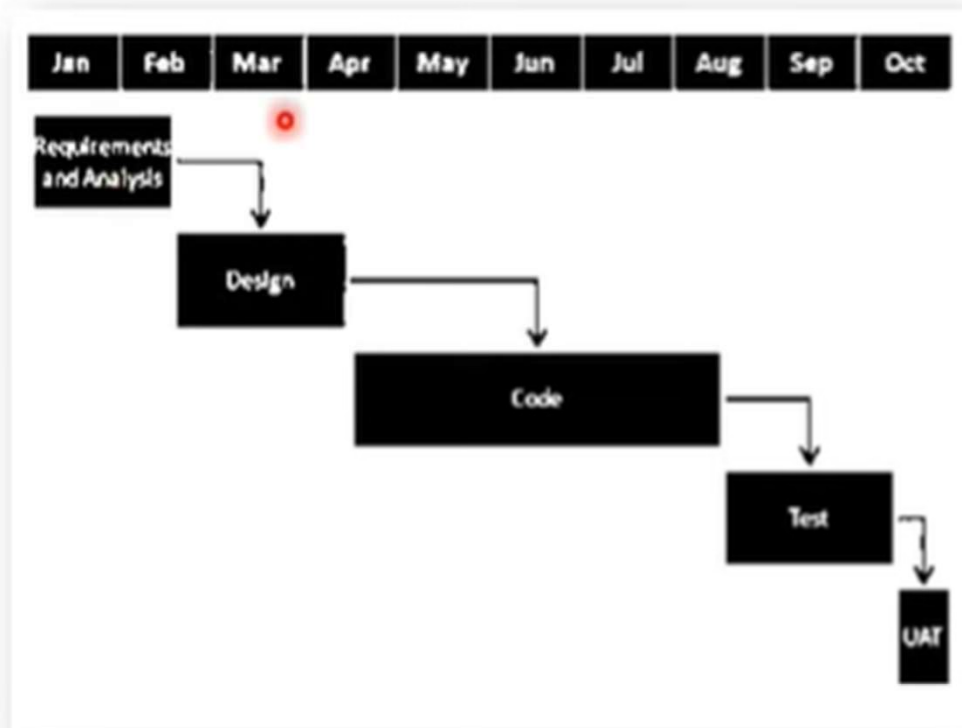
Waterfall Model

Agile Model

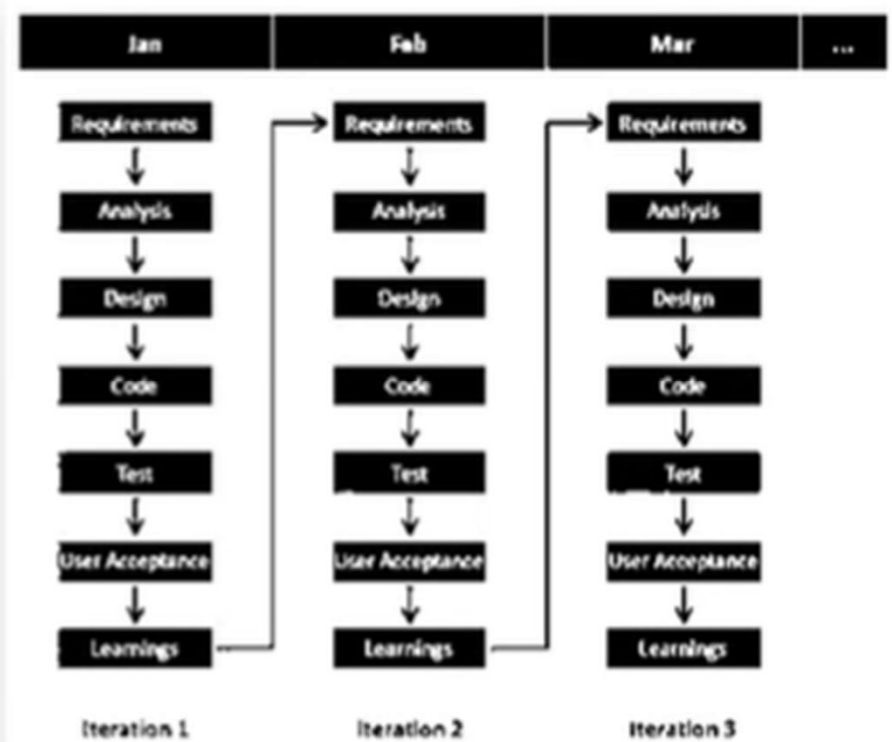


Traditional VS Agile Model Working with Example

For Example:



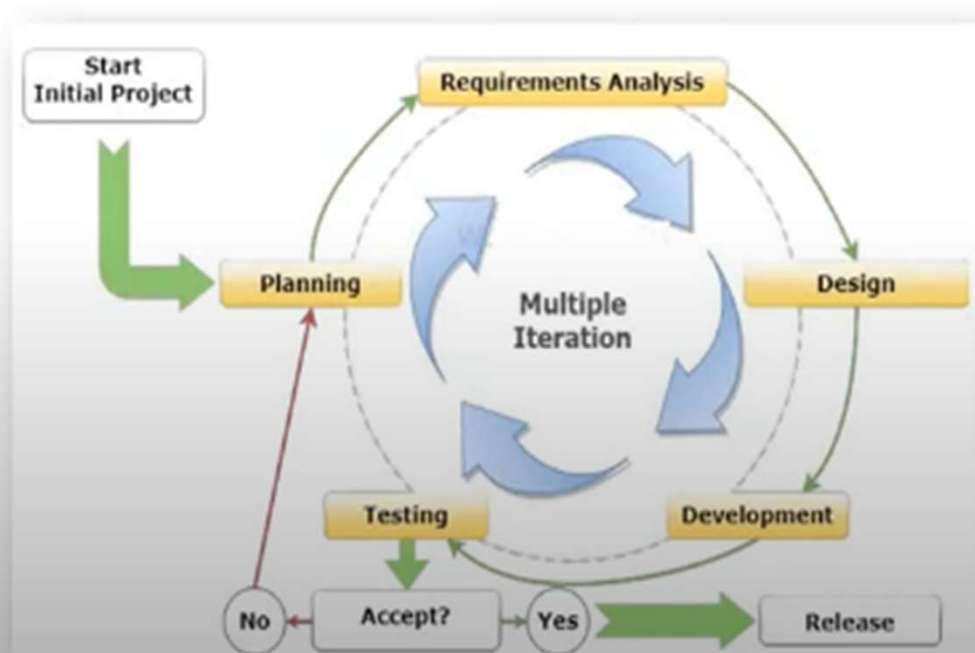
**Waterfall Model
Time Span**



**Agile Model
Time Span**

When to use the Agile Model?

1. When project size is large.
2. When frequent changes are required.
3. When a highly qualified and experienced team is available.
4. When a customer is ready to have a meeting with a software team all the time.
5. Projects with flexible timelines and budget



Agile Principles

1. Highest priority is to satisfy the customers to early & continue delivery of software.
2. Being flexible about changing requirements at any point of development.
3. Working on frequent and short deliveries like couple of weeks or months with preference.
4. Transparency between business people and developers and requires them to work together.
5. By providing a better productive environment and providing them with all the support, motivation it leads to better productivity.
6. Face to face communication as the most effective way to communicate between customer & development team.
7. Continuous attention towards effective designing and technical excellence through following optimal code standard.

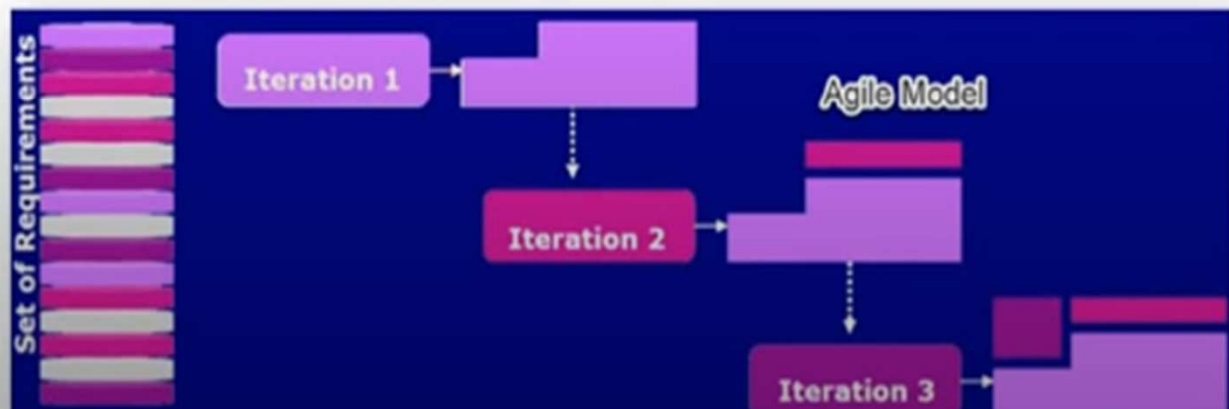
Agile Principles

8. It promotes sustainable development by developers, users and sponsors should work together.
9. Continuous attention to technical excellence and good design enhances agility.
10. Simplicity is the art of maximum result & less hard work by removing unnecessary tasks and prioritizing activities.
11. The best architectures, requirements, and designs emerge from self-organizing & experience teams.
12. For developing effective software, regular analysis and work on improving the overall delivery or the development process.



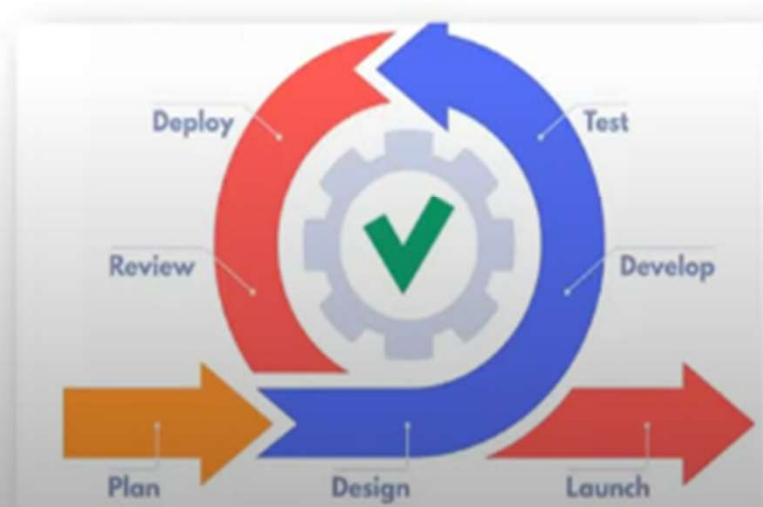
Advantages of Agile Model

1. Supports customer involvement and customer satisfaction
2. Strong communication of the software team with the customer.
3. Little planning required
4. Efficient design and fulfils the business requirement.
5. Anytime changes are acceptable.
6. Provides a very realistic approach to software development
7. Updated versions of functioning software are released every week.
8. It reduces total development time.



Disadvantages of Agile Model

1. Due to the lack of proper documentation, once the project completes and the developers allotted to another project, maintenance of the finished project can become a difficulty.
2. Depends heavily on customer interaction, so if customer is not clear, team can be driven in the wrong direction.



What is Software Requirement?

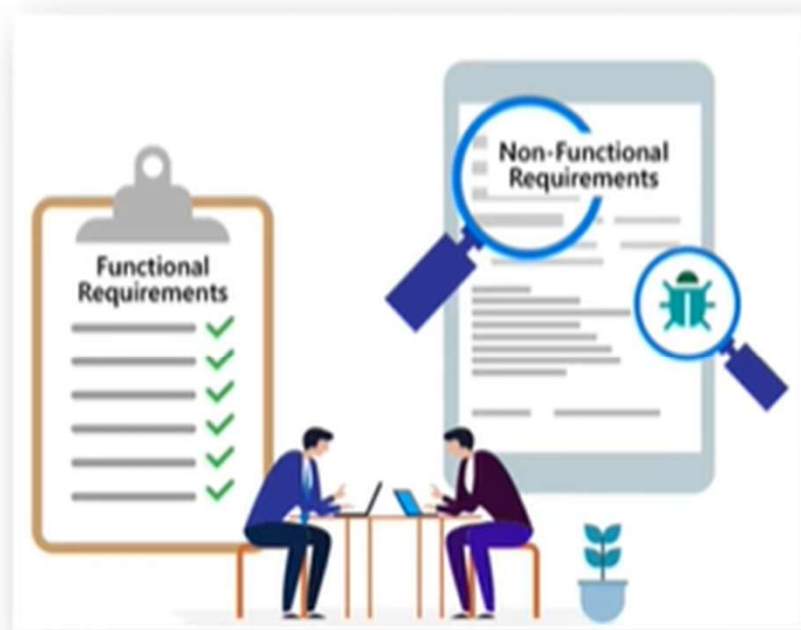
➤ Definition:

- According IEEE Standard, Software Requirement is a condition needed by a user to solve a problem or achieve an objective.
- Requirement should be clear, correct and well-defined.

➤ Types of Requirements:

There are two types of requirements:

1. Functional Requirements
2. Non Functional Requirements



Difference between Functional & Non Functional Requirements

No.	Functional Requirements	Non-Functional Requirements
1	Help to understand the <u>functions of the system</u> .	Help to understand the <u>system's performance</u> .
2	<u>Mandatory</u> requirements.	<u>Not mandatory</u> requirements.
3	They are <u>easy to define</u> .	They are <u>hard to define</u> .
4	It concentrates on the <u>user's requirement</u> .	It concentrates on the <u>expectation of the user</u> .
5	These requirements are specified <u>by the user</u> .	These requirements are specified by the <u>software developers, architects and technical persons</u> .

Difference between Functional & Non Functional Requirements

No.	Functional Requirements	Non-Functional Requirements
6	Describe <u>what the product does</u>	Describes <u>how the product works</u>
7	There is <u>functional testing</u> such as API testing, system, integration, etc.	There is <u>non-functional testing</u> such as usability, performance, stress, security, etc.
8	<u>First Completion of Functional requirements</u> allows the system to perform non-functional requirements.	While system <u>will not work only with non-functional requirements.</u>

Examples

➤ Examples of Functional Requirements:

1. In hospital management system, a doctor should be able to retrieve the information of his patients.
2. Search option given to user to search from various invoices.
3. User should be able to mail any report to management.
4. Only Managerial level employees have the right to view banking revenue data.

➤ Examples of Non-Functional Requirements:

1. Security, Logging, Storage, Configuration, Performance, Cost, Interoperability, Flexibility, Disaster recovery, Accessibility
2. Employees never allowed to update their salary information. Reported to the administrator.
3. A website should be capable enough to handle 20 million users with affecting its performance.
4. The software should be portable. Moving from one OS to other OS does not create any issue.

What is Requirement Engineering?

- The process to gather the software requirements from customers, analyze and document them is known as requirement engineering.
- **Requirement engineering provides the appropriate mechanism to understand:**
 - Customer desires / wishes,
 - Analyzing Customer need
 - Assessing feasibility
 - Negotiating a reasonable solution
 - Specifying the solution clearly
 - Validating the specifications and managing the requirementsas they are transformed into a working system



Users VS System Requirements

No.	Users Requirements	System Requirements
1	Written for <u>Customers</u> .	Written for <u>Implementation Team</u> .
2	In <u>Natural language</u> .	In <u>Technical Language</u> .
3	Describe the <u>services & features provide by system</u> .	Describe the <u>detail description</u> of services, features & complete operations of system.
4	May include <u>diagrams & tables</u> .	May include <u>system models</u> .
5	Understandable by <u>system users</u> who don't have technical knowledge.	Understandable by <u>implementation team</u> who have technical knowledge.
6	For Client Managers, System Users, Contractor Managers, System Architects	For System Architects, Software Developers, Client Engineers, System Users

Tools in Requirement Engineering

1. Observation report
2. Questionnaire (survey , poll)
3. Use cases
4. User stories
5. Requirement workshop
6. Mind mapping
7. Role playing
8. Prototyping

Available Softwares:

1. Xebrio – Requirements Management Software
2. Jira for requirements engineering
3. Doc Sheets
4. RequirementsHub
5. IBM Engineering Requirements Management DOORS
6. Modern Requirements
7. SpiraTeam

Establishing Ground Work

1. Identifying the stakeholders:

- Any person who benefits directly or indirectly from the system being developed is a stakeholder.
- **Examples:** Business operations managers, product managers, marketing people, internal and external customers, end-users, consultants, product engineers, software engineers, developers, testers and support/maintenance engineers are the usual stakeholders.
- Each stakeholder sees the system differently, gains different benefits when the system is successfully developed.
- They also faces different risks if the development effort fails.

Establishing Ground Work

2. Recognizing Multiple Viewpoints:

- Because there are so many different stakeholders, the system's requirements will be examined from various perspectives.
- Each of these stakeholders will contribute data to the requirements engineering process.
- As information is gathered from multiple viewpoints, collecting requirements may be inconsistent or contradictory.
- You should categories all stakeholder information so that decision-makers can select an consistent set of system requirements for the system.

Establishing Ground Work

3. Working toward Collaboration:

- If there are five stakeholders involved in a software project, there may be five different opinions on the set of requirements.
- Customers must work together with software development team to create a successful system.
- A requirements engineer's job is to identify areas of commonality requirement as well as areas of conflict requirements.
- At the end project head & requirement engineer may decide on which requirements are accepted.

Establishing Ground Work

4. Asking the first questions: (For identifying stakeholders)

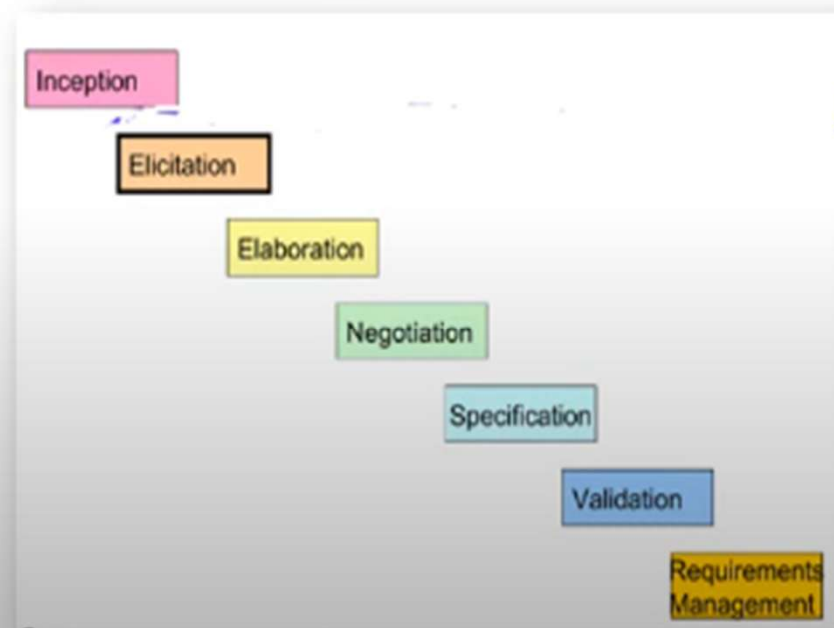
- The first set of questions are focuses on the customers, stakeholders as well as the overall project goals and benefits.
 1. Who is the person or organization behind the request for this work?
 2. Who will make use of the solution?
 3. What is the economic value of a successful solution?
 4. Is there a different source for the solution you require?

Requirement Engineering Tasks

- The process of collecting the software requirements from the client and then understanding, analysis, and documenting it is called requirement engineering.
- Requirement engineering constructs a bridge for design and construction.

➤ Requirement Engineering consists of seven different tasks:

1. Inception
2. Elicitation
3. Elaboration
4. Negotiation
5. Specification
6. Validation
7. Management



1. Inception

- This is the first phase of the requirements analysis process or starting of project.
- Requirement engineering asks a set of questions to establish a software process.
- They understand basic details, aim, goal of the project & find out the solution.
- They identify stakeholders & who want the solution.
- They understand nature of solution.
- Enhance collaboration between customer & developer.



2. Elicitation

- This phase focuses on gathering the requirements from the stakeholders.
- The right people must be involved in this phase, no space for mistake.

➤ **The following problems can occur in the elicitation phase:**

1. **Problem of scope:** The customer give the unnecessary technical details rather than clarity of the overall system objective. Not possible to implement details have given.
2. **Problem of Understanding:** Not having a clear understanding between the developer and customer. Sometimes the customer might not know what they want or the developer might misunderstand one requirement for another.
3. **Problem of Volatility:** Requirements changing over time can cause difficulty in developing project. It can lead to loss and wastage of resources and time.

3. Elaboration 4.Negotiation

3. Elaboration:

- It takes the requirements that have been gathered in the first two Inception & Elicitation phases and expand them.
- Its main task is developing model & prototype of software using functions, feature and constraints of a software using different tools.



4. Negotiation:

- Negotiation is between the developer and the customer about limited availability of resources, delivery time, project cost & overall estimation of project.
- Customers are asked to prioritize the requirements for development & also analyze any conflict has occur.

5. Specification 6. Validation



5. Specification:

- In this task, the requirement engineer constructs a final work product.
- The work product is in the form of software requirement specification document.
- It collect all user, system, functional & non-functional requirements.
- ER, DFD, Data dictionary model used in this phase.
- A software specification document is submitted to the customer in a language that he/she will understand, to give a glimpse of the working model.

6. Validation:

- It checking for errors and debugging in final work product or SRS document.
- It check all the requirements have been stated and met correctly as per stakeholders.
- It check any missing information or want to add any additional information.



7. Requirement Management

- Requirements management is a set of activities where the entire team takes part in identifying, controlling, tracking, and establishing the requirements for the successful and smooth implementation of the project.
- It track on new additional requirement implementation which does not affect on overall system.
- Based on this phase, the working model will be analyzed carefully and ready to be delivered to the customer.



Summary

➤ **Requirement Engineering consists of seven different tasks:**

- 1. Inception** (Better Understanding)
- 2. Elicitation** (Clear Understanding of Requirements)
- 3. Elaboration** (Refinement & Modelling)
- 4. Negotiation** (Settlement & Conflicts)
- 5. Specification** (SRS Document)
- 6. Validation** (Validates Requirements)
- 7. Management** (Manage changing requirement)